

Connected Causal Kernels and Commuting-Layer Search for Reidemeister-III Unlocking

Single-exponential local search and a sharper route
toward quasi-polynomial unknot recognition

Research contribution prepared for Vladimir Reshetnikov's ProveIt project

8 October 2026

Abstract

We study the exact local subproblem of deciding whether at most k Reidemeister-III moves can expose a crossing-decreasing Reidemeister-I or Reidemeister-II move in an N -crossing classical knot diagram. The inspected ProveIt implementation bounds a related support-aware search by $N^{b+O(1)}(Ck)^k$, where b is the number of independent births. We first replace chronological interleavings by commuting layers and obtain $N^{b+O(1)}2^{O(k)}$. A state-dependent version of the usual heap construction admits an injective encoding by bounded-slot forests; the exact forest count is $\frac{b}{j} \binom{dj}{j-b}$.

A second argument removes the birth parameter for the unlocking problem. The backwards causal cone of one terminal reduction has a set of at most $3k + 2$ core crossings connected in the *initial* projection graph, even though the intervening moves change that graph. Full read/write footprints are retained. Enumerating these connected cores and applying the layer search gives a deterministic $\text{poly}(N, k)2^{O(k)}$ algorithm, with polynomial workspace, for every bounded unlocking instance. A deliberately loose explicit bound is $\text{poly}(N, k)2^{23k+11}$.

This strengthens a component of the recognition pipeline, not the general recognition theorem. If the actual deterministic simplification run has unlocking budget g and residual core size r , exact recognition costs $\text{poly}(N, g)2^{O(g)} + 2^{O(r)}$; thus $g, r = O(\log^2 N)$ suffice for the specific target $N^{O(\log N)}$. No uniform such bounds are established. The accompanying standard-library Python kernels, certificate verifier, 27 unit tests, exhaustive Boolean comparisons, and actual-diagram audits make the local claims reproducible. Paired timings show fewer interleavings but no demonstrated wall-clock improvement on the measured knot kernels; no production default change is recommended.

Contents

1	Scope, baseline, and the contribution	2
1.1	Relation to established work	2
2	Bounded unlocking and the local action model	3
3	State-dependent commuting layers	4
3.1	What the normal form does and does not identify	5
3.2	Enumeration	5
4	The bounded-slot forest count	5
5	RIII footprints: 18 darts, nine crossings, 36 successors	7
6	The backwards cone of one terminal reduction	9
6.1	A sharper invariant: connected cores rather than neighborhoods	10
7	An unconditional single-exponential unlocking algorithm	12
7.1	Bit complexity and implementation costs	13
7.2	Algorithmic skeleton	14
8	Consequences for the complete recognition pipeline	14
8.1	Where the universal theorem is still missing	15
9	Executable design, certificates, and integration safety	16
9.1	Package components	16
9.2	One global copy for a disjoint layer	16
9.3	Certificate contract	16
9.4	Safe opt-in adapter	17
10	Validation and measured performance	17
10.1	Tests and exhaustive checks	17
10.2	Actual-diagram audit	18
10.3	Paired kernel timings, including negative results	18
11	Limits, proof obligations, and failure modes	19
12	Further research questions and concrete work packages	20
12.1	Reduce the single-exponential base without weakening completeness	20
12.2	Turn reduced interleavings into measured speed	21
12.3	Bridge the local theorem to stronger recognition classes	21
12.4	Relate local search to hierarchy and complexity theory	22
12.5	Immediate integration decision	22
13	Conclusion	22
A	Reproduction and integration instructions	23
B	Proof-dependency and evidence ledger	23

1 Scope, baseline, and the contribution

The computational target is exact recognition of a knot from a classical one-component diagram, with an eventual worst-case bound of $N^{O(\log N)}$. Here N denotes the number of crossings; logarithms in complexity comparisons are to base two. In the broader literature, “quasi-polynomial” can mean $\exp((\log N)^{O(1)})$. We retain the more specific $\exp(O(\log^2 N))$ target used in this project.

The relevant inspected repository material is the integrated causal RIII search and its audit in `synthesis/causal_r3.tex`, together with `fast/fastunknot/causal_r3.py`, the dart operations in `simplify.py`, and the diagram conventions in `diagram.py` [1, 2, 3, 4]. The implementation reads and analysis are recorded in `SOURCES.json`. The directly pinned source revision is

34f2689e96e04a13a32d133173b5e41a34320086.

The source files fetched just before this pin are additionally identified by blob hashes; they are not silently treated as a complete checkout.

The existing causal method distinguishes an initial collection of independent birth moves from later moves intersecting accumulated support. Its analysis has the shape $N^{b+O(1)}(Ck)^k$. It also explains why physical-state-only memoization and indiscriminate inverse pruning can destroy completeness with a birth constraint. Its integrated trace representation identifies an actual triangular face rather than merely a triple of crossings [1]. Those are sound prerequisites to preserve, not replace.

Our first change is to enumerate *layers of commuting moves*, removing a factorial-scale interleaving overhead. Our second change uses the fact that unlocking has a *local terminal witness*: a specific empty monogon or eligible bigon. This allows us to search a connected support of size linear in k , rather than paying N^b for unrelated starting regions.

Claim	Scope	Status here
Commuting-layer bound	General bounded-local action systems with the stated commutation contracts	Proved; executable enumeration
Connected-support bound	Classical knot diagrams; RIII moves followed by one RI/RII test	Proved; executable region search
Quasi-polynomial recognition	Actual run has $g, r = O(\log^2 N)$	Conditional corollary, not a uniform theorem
Production speedup	Current complete <code>fastunknot</code> pipeline	Not measured; no default switch proposed

The main theorem is stronger than a promise about a small birth number: *every* instance of the bounded unlocking problem is covered. It is weaker than general recognition because a diagram can require a much larger unlocking budget, or the permitted crossing-nonincreasing strategy can stall. These quantifiers must not be conflated.

1.1 Relation to established work

Heaps of pieces and Cartier–Foata normal forms are classical ways to identify words differing only by commutations [5, 6]. The contribution here is not the invention of those normal forms. It is the

explicit treatment of state-dependent local actions, the bounded-slot count, the dart constants, and the connected initial-support argument needed for this recognition pipeline.

Legrand-Duchesne, Rai, and Tancer study the number of moves required to *untangle the whole diagram*, including the parameter called defect, and establish $W[P]$ -completeness for that parameter [7]. Our parameter counts only RIII moves in one episode ending in *one* reduction. Their parameter, their terminal condition, and their permitted global histories are different. In particular, local fixed-parameter tractability proved here does not decide whether an arbitrary diagram admits a globally optimal untangling sequence.

Lackenby’s July 2026 preprint develops an incompressibility and hierarchy algorithm with an application to the unknot [8]. Section 9 discusses the remaining cost analysis and the precision of its algorithmic steps. The present paper does not implement those hierarchy operations or supply their missing estimates. It addresses a rigorously specified local kernel in the existing alternative pipeline. For a complete fallback we may use reduced Khovanov homology, whose unknot detection is a theorem of Kronheimer and Mrowka [9]; Bar-Natan’s computational work provides a principal antecedent for practical complex simplification [10].

2 Bounded unlocking and the local action model

Definition 2.1 (Bounded RIII unlocking). Given a connected classical one-component diagram D and a nonnegative integer k , the problem $\text{Unlock}_{\text{RIII}}(D, k)$ asks whether a legal sequence of at most k RIII moves produces a diagram admitting a crossing-decreasing RI or RII move. The terminal reduction need not yet be performed. There are no crossing-increasing moves or crossing deletions within the RIII sequence.

An initially available reduction gives a zero-move witness. An empty diagram has no crossing-decreasing move and therefore has a negative answer to this *local* question; the outer recognizer of course knows that the empty knot diagram is an unknot. A completed negative local answer says only that the prescribed episode does not exist. A resource cap gives an inconclusive answer even to the local question.

For the abstract layer theorem, use a fixed finite site set X . A state assigns local data to those sites, possibly subject to global invariants. A legal action a at a state has a unique key, a nonempty conservative footprint $F(a) \subseteq X$, and a transition to a new state. The key is a stable identifier of that particular local move, not merely a name for the set of vertices it involves.

Definition 2.2 (Locality and commutation contracts). The action system satisfies the following requirements.

- (i) The footprint contains all sites whose data determine legality, all data read to perform the action, and all data changed by the action.
- (ii) Disjoint actions preserve one another’s legality, keys, footprints, and effects. They commute as transitions. In a legal chronological trace, adjacent occurrences with disjoint footprints can be swapped in either direction, retaining the same occurrence data.
- (iii) Legal actions and their footprints are deterministically enumerable. State descriptions, enumeration, key comparison, transitions, and target tests when used have costs polynomial in the input size and in the trace-length bound.

- (iv) A bound d is available such that, after a layer of disjoint actions, at most d legal actions intersect the footprint of any specified action of that preceding layer.

Requirement (ii) is deliberately stronger than saying that two initially legal operations happen to have the same final result in one example. It includes the exchange property for occurrences in a trace. Read/write locality with stable footprint descriptions supplies this property. An implementation using an insufficient read set does not satisfy the theorem.

One general way to obtain (iv) is to bound every footprint by f sites and every site's incidence with legal action footprints by B . Then $d = fB$ is valid. We will obtain a better RIII-specific value $d = 36$.

Definition 2.3 (Births and dependency heights). For a chronological trace $a_1 \cdots a_m$, write $F_i = F(a_i)$ for the footprint at its occurrence. Occurrence j is a birth when $F_j \cap \bigcup_{i < j} F_i = \emptyset$. Its dependency height is

$$h_j = 1 + \max(\{h_i : i < j, F_i \cap F_j \neq \emptyset\} \cup \{0\}).$$

The ℓ th layer is the set of occurrences having height ℓ , sorted by their action keys.

Notice the distinction between accumulated support and the previous layer. The old birth-front search branches on the former. The new normal form requires every action after the first layer to meet the *immediately preceding* layer. That stronger condition is a consequence of dependency height, not a heuristic restriction to the neighborhood of the last move.

3 State-dependent commuting layers

Theorem 3.1 (Dynamic layer normal form). *Under the locality and commutation contracts, every legal trace has a legal layered representative with the same length, endpoint, and occurrence footprints. Actions within each layer are pairwise footprint-disjoint. Every action in layer $\ell > 1$ meets an action in layer $\ell - 1$. The first layer consists exactly of the original births. Conversely, a legal sequence of pairwise-disjoint layers with this previous-layer condition is the height-layer representative of its own chronological expansion.*

Proof. If $i < j$ and $F_i \cap F_j \neq \emptyset$, then $h_j \geq h_i + 1$. Consequently two occurrences of equal height have disjoint footprints. If $h_j = \ell > 1$, the maximum in the defining recurrence is attained at an earlier occurrence of height $\ell - 1$. Thus the claimed previous-layer intersection exists.

Orient every intersecting pair of occurrences from its earlier member to its later member. This directed graph is acyclic because chronological indices increase along its edges. The original word is a linear extension of its transitive closure. Ordering occurrences first by height and then by key is another linear extension. Two linear extensions of a finite poset can be connected by adjacent swaps of incomparable elements: move the first element of the desired order leftwards across its predecessors in the current order, and repeat on the remaining suffix. Incomparable occurrences here have disjoint footprints. Requirement (ii) therefore makes each swap legal and preserves every involved footprint and key, including when the action catalogue is state-dependent. The sorted word is a legal trace with the same endpoint.

Height one means that there is no earlier intersecting occurrence, exactly the definition of birth. Since equal-height keys are distinct in their common pre-layer state, the canonical order is unambiguous.

For the converse, proceed by layers. Actions in the first layer have no earlier intersecting action. An action in the next layer has an intersecting predecessor in the previous layer and no earlier action at a greater height. Equal-layer actions are disjoint. Its height is consequently exactly its layer index. Induction proves the assertion. $\square$

3.1 What the normal form does and does not identify

The normalization removes permutations of independent occurrences. It does not identify all words with the same physical endpoint: inverses, different local relations, and different histories can still lead to the same state. The algorithm is not a shortest-path quotient by physical-state equality. This is intentional, since support history can matter to the restricted birth problem.

For example, take four Boolean sites initially zero and the following legal involutions, as in the repository’s inverse-pruning audit [1]:

Action	Flipped bits	Guard	Footprint
A	0, 1	$x_2 = x_3 = 0$	$\{0, 1, 2, 3\}$
B	2	$x_0 = 0$	$\{0, 2\}$
C	3	$x_1 = 0$	$\{1, 3\}$

For the terminal predicate $x_2 = x_3 = 1$, the trace BC has two births, whereas $AABC$ has one. Its layers are $\{A\}, \{A\}, \{B, C\}$. Deleting the neutral pair AA changes the birth count. The implementation therefore retains immediate inverses. The root-free local algorithm also retains them, avoiding a second set of pruning assumptions.

3.2 Enumeration

At the initial state enumerate pairwise-disjoint nonempty subsets of legal actions, optionally imposing a first-layer size bound b . At subsequent states, enumerate the legal actions meeting the previous layer’s footprint union, then enumerate their nonempty pairwise-disjoint subsets. Charge a layer its number of actions, not one unit. Stop extending at total charge k and test the target at every completed prefix, including the empty one.

Testing only at completed layers is complete for bounded reachability. Starting with any successful chronological prefix, Theorem 3.1 produces a completed-layer trace reaching the same endpoint. There is no need to insert arbitrary partial executions of a chosen layer into that trace’s search path: those smaller layers are already independently enumerated by the algorithm.

An explicit depth-first stack holds one suspended subset iterator per active level. It never stores a global table of all reached states. Resource exhaustion propagates as an exception distinct from complete bounded failure. The reference implementation uses immutable states, so a failed or interrupted search cannot corrupt its input diagram.

4 The bounded-slot forest count

The crude chronological estimate gives $O(k)$ locally available extensions at each of k stages and hence a factor $(Ck)^k$. Merely replacing that estimate by a verbal appeal to commutation would be insufficient. We give an explicit encoding that also handles state-dependent actions.

Lemma 4.1 (Forest encoding). *Fix an initial layer of $b \geq 1$ actions. Layered traces with j occurrences in total inject into ordered forests with b rooted trees, where every node has d distinguished child slots, each either empty or occupied by one child. The same upper bound covers attempted last layers rejected for internal footprint overlap, provided all preceding layers are legal.*

Proof. After a legal layer, order its actions by key. Every candidate for the next layer intersects at least one parent action. Assign it to the first such parent. For each parent, sort its assigned candidates by key and number them from one. There are at most d of them by contract (iv). For each selected child, record the corresponding slot at its assigned parent. Repeat for all layers; the roots are the prescribed initial actions in canonical order. Each nonroot has one parent in the previous layer, so the result is an ordered d -slot forest.

To prove injectivity, reconstruct breadth-first. The root layer and input state determine the first post-layer state. That state determines the candidate catalogue, parent assignments, and slot meanings. The occupied slots uniquely determine the next selected actions. Their keys determine canonical order and their transitions determine the next state. Continue by induction. No assumption of a static global alphabet is used.

If the last attempted subset has an internal conflict, do not apply it. Its candidate catalogue and assignments are still determined by the legal preceding state. The same slot data uniquely reconstruct that attempted subset. Thus rejected final attempts are covered without needing a post-conflict state. $\square$

Lemma 4.2 (Exact forest enumeration). *The number of ordered d -slot forests with b roots and $j \geq b$ nodes is*

$$F_d(j, b) = \frac{b}{j} \binom{dj}{j-b}. \quad (1)$$

For $d \geq 1$, this is at most $(ed)^j$.

Proof. Let $T(z)$ count nonempty rooted trees by their number of nodes. Choosing which of the d child slots are filled gives $T = z(1 + T)^d$. A forest of b ordered roots has generating function T^b . The coefficient form of Lagrange inversion yields

$$[z^j]T(z)^b = \frac{b}{j} [u^{j-b}](1+u)^{dj},$$

which is (1). This use of formal power series involves only finite integer coefficients in each degree.

For the inequality, put $q = j - b$. If $q = 0$ the count is one. If $q > 0$, use $\binom{dj}{q} \leq (edj/q)^q$ and $b/j \leq 1$. Writing $t = q/j \in (0, 1]$,

$$\log((edj/q)^q) = j t \log(ed/t) \leq j \log(ed).$$

Indeed the derivative of $t \log(ed/t)$ is $\log(d/t) \geq 0$ on that interval. This proves the bound, including $d = 1$. $\square$

Theorem 4.3 (Birth-bounded single-exponential search). *Suppose the initial state has M legal actions and at most b births are permitted. The layer search through k actions has cost at most a polynomial factor times*

$$1 + \sum_{q=1}^{\min(b, k, M)} \binom{M}{q} \sum_{j=q}^k \frac{q}{j} \binom{dj}{j-q}. \quad (2)$$

In particular its cost is

$$\text{poly}(N, k) \left(1 + \sum_{q=1}^{\min(b, k, M)} \binom{M}{q} \right) (ed)^k. \quad (3)$$

For fixed d and $M = O(N)$, this is $N^{b+O(1)}2^{O(k)}$. The search can be implemented in polynomial workspace.

Proof. Choose the root subset, charging even subsets that fail the disjointness test. For every legal root subset use Lemmas 4.1 and 4.2. A yielded prefix and an attempted extension are both bounded by forests of at most k nodes. Candidate enumeration, subset materialization, compatibility checking, and transitions cost a polynomial factor per such object under contract (iii). Thus counting only successful paths does not hide an uncontrolled cost of rejected subsets. The extra sum over j is absorbed in $\text{poly}(N, k)$.

At most k nonempty layers are active on a depth-first path. Each frame stores a polynomial-size state, candidate collection, subset iterator, and history. All integers describing keys, site indices, and counts have polynomially bounded bit length in the input and k . No exponentially large visited-state cache is required. $\square$

Remark 4.4 (An elementary backup bound). For layer sizes $\ell_1, \dots, \ell_h$ summing to j , a next layer has at most $d\ell_i$ candidates. Ignoring size compatibility, their subsets are at most $2^{d\ell_i}$. Multiplying over nonfinal layers gives at most 2^{dj} choices, and the number of compositions of j is at most 2^{j-1} . This already proves a $2^{O(dk)}$ bound for fixed roots and fixed d . The forest argument sharpens the base and gives the auditable coefficient formula; the single-exponential conclusion does not depend on subtle analytic estimates.

5 RIII footprints: 18 darts, nine crossings, 36 successors

The diagram representation has four counterclockwise darts at each crossing. Darts $4c$ and $4c + 2$ are the under-strand ports; darts $4c + 1$ and $4c + 3$ are the over-strand ports. The fixed-point-free involution α pairs the ends of each edge. Define

$$\text{rot}(4c + j) = 4c + (j + 1 \bmod 4), \quad \phi(d) = \text{rot}(\alpha(d)).$$

Faces are cycles of ϕ . A legal RIII candidate is a triangular face $T = (d_0, d_1, d_2)$ on three distinct crossings satisfying the over/under condition: at least one side is over at both ends. Its identity is the sorted triple of its actual face darts. The crossing triple alone is not a unique move identifier [3, 1].

Let $D(T)$ be all twelve darts at its three crossings and let

$$F(T) = D(T) \cup \alpha(D(T)), \quad V(F) = \{ \lfloor d/4 \rfloor : d \in F \}. \quad (4)$$

This is a conservative read/write footprint. Even when two exterior strands are directly connected, the formula includes their partner darts. It avoids an unjustified assumption that the six exterior ports have six distinct remote neighbors. Write $Q(T)$ for the set of the three core crossings themselves. The distinction between $Q(T)$ and $V(F(T))$ will be important: a small core is not a safe replacement for a read/write footprint.

Lemma 5.1 (Local support bounds and closure). *A legal nondegenerate RIII move has $|F(T)| \leq 18$ and $|V(F(T))| \leq 9$. The set $F(T)$ is α -closed before the move, is unchanged as a set by the move,*

and is closed under the new edge pairing. Every changed pairing entry lies in $F(T)$. Its crossing support induces a connected subgraph of the current projection graph.

Proof. The three sides of the triangular face use six distinct darts in $D(T)$, already paired to one another. Therefore at most the other six darts of $D(T)$ have partners outside $D(T)$; adding those partners gives at most $12 + 6 = 18$ darts. At the crossing level there are the three central crossings and at most six exterior endpoint crossings, for a total of nine. Coincidences only reduce these bounds.

Closure before the move follows immediately from $\alpha^2 = 1$: $\alpha(D \cup \alpha(D)) = D \cup \alpha(D)$. The move interchanges the order in which the three strands meet, retaining the twelve central ports and retaining precisely the same exterior partner darts. It reconnects pairings among this set, never introducing another exterior endpoint. Hence its footprint and closure are preserved, and no entry outside it changes. Equivalently this can be checked directly in the pairing patch of `triangle_patch`.

The three core crossings form a triangle in the projection graph. Every additional crossing in $V(F)$ is joined to one of those crossings by an edge whose dart lies in D . Consequently the induced support subgraph is connected. Loops and repeated edges do not invalidate this argument. $\square$

Lemma 5.2 (Site incidence). *At any state, each dart belongs to at most eight candidate RIII footprints. The total number M of candidates is at most $\min(N + 2, \lfloor 4N/3 \rfloor)$. If only moves with $V(F) \subseteq S$ are eligible, their number is at most $\lfloor 4|S|/3 \rfloor$.*

Proof. A dart x can lie in $F(T)$ only if x or $\alpha(x)$ lies in $D(T)$. Thus a candidate must use the crossing of x or the crossing of $\alpha(x)$. At a fixed crossing, at most four face corners can belong to triangular faces. Counting each candidate at most once per relevant corner gives the bound eight.

A connected spherical four-valent projection with N vertices has $N + 2$ faces by Euler's formula. Distinct triangular face cycles use disjoint sets of three darts, so their total number is also at most $4N/3$. For an eligible candidate supported in S , its three face darts all belong to crossings of S . Counting those darts proves the local bound. Over/under and nondegeneracy tests can only discard candidates. $\square$

Using only these bounds would give $d = 18 \cdot 8 = 144$. Closure gives a substantially smaller constant.

Lemma 5.3 (Thirty-six next-layer candidates). *After executing a layer of pairwise footprint-disjoint RIII moves, at most 36 legal RIII candidates intersect the footprint F of any specified parent move from that layer.*

Proof. By Lemma 5.1, F is unchanged and closed under the post-layer pairing α : disjoint sibling moves do not change entries in F . For a candidate with core D' and footprint $F' = D' \cup \alpha(D')$,

$$F' \cap F \neq \emptyset \iff D' \cap F \neq \emptyset.$$

The reverse implication is immediate. For the forward implication, an intersection at $\alpha(y)$ with $y \in D'$ implies $y \in F$ by closure. Such a candidate uses a crossing in $V(F)$, which has size at most nine. There are at most four triangular-face corners at each such crossing. Thus at most $9 \cdot 4 = 36$ candidates can occur. $\square$

Corollary 5.4 (RIII layer budget). *The complete search for RIII traces of at most k actions and at most b births has cost $N^{b+O(1)}2^{O(k)}$. More precisely, (2) applies with $d = 36$ and the initial candidate count from Lemma 5.2.*

No planar separator theorem is needed for this corollary. The structural facts used are bounded valence, fixed ports, local commutation, and a constant-size pairing update. The sharper nine-crossing support will be used to eliminate the global choice of births.

6 The backwards cone of one terminal reduction

The next step depends on the terminal predicate being local. For a selected RI test, take all four darts of its crossing and their partners; this footprint has at most six darts and three crossing vertices. For a selected RII test, take the eight darts of its two crossings and their partners; the bigon's two sides already account for four internal darts, leaving at most four remote endpoints. This footprint has at most twelve darts and six crossing vertices. Denote it by F_ρ . It is closed under the current pairing and its crossing support is connected. It contains every pairing entry needed to recognize the selected legal face and its over/under condition. The test itself is read-only.

Lemma 6.1 (Terminal ancestor cone). *Let $a_1 \cdots a_m$ be a legal trace ending with a particular legal local test ρ . Scan the occurrences backwards, starting with active set F_ρ . Retain an occurrence exactly when its footprint intersects the active set, and then add its footprint to that set. The retained chronological subsequence is legal from the same input, makes ρ legal, and has the same retained occurrence footprints. The intersection graph of retained footprints together with F_ρ is connected.*

Proof. An omitted occurrence is disjoint from every later retained occurrence and from the terminal test footprint. It can be moved to the right of all those retained occurrences by allowed adjacent exchanges. One may formalize this by repeatedly taking the rightmost omitted occurrence, commuting it across the retained suffix, and regarding it as occurring after the test. Earlier omitted occurrences are treated in the same way. The resulting trace begins with the retained subsequence, followed by the test; omitted actions occur afterwards and can be dropped. All retained local data and the test's local data are unchanged by the exchanges.

Each retained footprint intersects the terminal footprint or a footprint retained later in the backwards scan. Starting with the terminal as a single connected node, adjoining retained nodes in scan order preserves connectedness of the intersection graph. $\square$

For a cone containing at most k RIII moves, its crossing union

$$S = V(F_\rho) \cup \bigcup_{a \text{ retained}} V(F(a)) \quad (5)$$

satisfies $|S| \leq 9k + 6$. A naive assertion that this region is connected in the initial diagram would still require proof: edges change during RIII. The following invariant is the missing step.

Lemma 6.2 (Initial-component confinement). *Consider a sequence of local rewrites on a fixed graph vertex set. Suppose each rewrite has a support contained in S , induces a connected subgraph of the current graph, and changes edges only between vertices of its own support. Then no rewrite creates an edge between different components of the initial induced graph $G_0[S]$. In particular, each rewrite support lies in one initial component. If a final connected test support and all rewrite supports have a connected intersection graph and union S , then $G_0[S]$ is connected.*

Proof. Partition S into the components of $G_0[S]$. Initially no induced edge joins different parts. Suppose this remains true before a particular rewrite. Its connected support must lie in a single part, since a path in the induced support would otherwise contain an edge joining two parts. Every changed edge has both endpoints in that support. Therefore no changed edge joins two parts and the invariant persists. This proves the first assertion by induction.

The final test support is also connected in the current induced graph, so it lies in one initial part. Intersecting supports cannot lie in distinct parts. The connected intersection graph forces all supports into the same part, and their union is S . Thus there is only one part. $\square$

Theorem 6.3 (Connected initial support for unlocking). *If $\text{Unlock}_{\text{RIII}}(D, k)$ is true and the input has no immediate RI/RII move, it has a witness of at most k RIII moves such that all move footprints and one terminal reduction footprint have crossing supports in a set S satisfying*

$$|S| \leq 9k + 6, \quad G_D[S] \text{ is connected.}$$

Here G_D is the initial projection graph, not a graph chosen after performing the witness.

Proof. Apply Lemma 6.1 to any successful trace and one terminal reduction. Work with the retained trace from the original input; omitted moves no longer participate in its state evolution. Lemma 5.1 gives connected crossing supports of size at most nine and changes only edges with endpoints in those supports. The terminal support has size at most six and is connected. Intersecting dart supports have intersecting crossing supports. Their intersection graph is therefore connected. Lemma 6.2 applies to the union in (5). The size estimate follows by summing the individual support bounds. $\square$

Remark 6.4 (Why this is not a geometric disk assumption). The induced region $G_D[S]$ may have cycles, holes in a planar picture, or several boundary components. The algorithm keeps the complete diagram and all external pairings. It does not cut out an unspecified disk or replace the complement by arbitrary boundary data. The full-support variant restricts the whole footprint to S . The sharper core variant below uses cores only to restrict which moves may be chosen, while retaining their entire read/write footprints and the full ambient state. Neither variant hides an unjustified tangle replacement.

6.1 A sharper invariant: connected cores rather than neighborhoods

The preceding support theorem is sufficient for single-exponential search, but it pays to enumerate exterior neighbors that are already determined by a much smaller set. For any crossing set C , define the dart star

$$D(C) = \{4c + j : c \in C, 0 \leq j < 4\}, \quad \mathcal{F}_\alpha(C) = D(C) \cup \alpha(D(C)).$$

Its crossing support is exactly the closed neighborhood $N_G[C]$ in the current projection graph. Here the notation $N_G[C]$ includes C itself.

Lemma 6.5 (Core contraction and star invariance). *Let a RIII move have core Q , and let C contain Q . The move preserves $\mathcal{F}_\alpha(C)$ as a set of darts, and it preserves the component partition of the induced graph $G[C]$. Furthermore, for any two nonempty core sets C, Q in the same state,*

$$\mathcal{F}_\alpha(C) \cap \mathcal{F}_\alpha(Q) \neq \emptyset \iff C \cap Q \neq \emptyset \text{ or an edge joins } C \text{ to } Q.$$

Proof. Before and after the move, the three vertices of Q contain the sides of a triangular face and hence form a connected subgraph. The move retains the same exterior endpoint darts and reattaches each to a port in Q . Edges with both endpoints outside Q are unchanged. Consequently, contracting Q to one vertex gives the same graph outside that vertex before and after the move, with the same incident exterior edges. This also holds after inducing on any set C containing Q . Since Q itself stays connected, the component partition of $G[C]$ is unchanged.

For star invariance, the full footprint $\mathcal{F}_\alpha(Q)$ is already contained in both the old and the new $\mathcal{F}_\alpha(C)$. Its set of darts is unchanged by Lemma 5.1. For a dart in $D(C) \setminus \mathcal{F}_\alpha(Q)$, the pairing is unchanged. A dart of $D(C)$ inside that footprint has both its old and new partner inside the same footprint. These observations account for every member of the old and new stars and prove their equality.

Both stars are pairing-closed. Their intersection is nonempty exactly when $D(Q)$ meets $\mathcal{F}_\alpha(C)$. A dart of $D(Q)$ either lies at a crossing of C , or is paired to a dart at a crossing of C . These are precisely the two alternatives in the equivalence. $\square$

Theorem 6.6 (Connected initial core of a terminal cone). *Under the hypotheses of Theorem 6.3, there is a successful retained trace of at most k RIII moves for which the union C of the move cores and one terminal reduction core satisfies*

$$|C| \leq 3k + 2, \quad G_D[C] \text{ is connected.} \quad (6)$$

The full dart union of its backwards cone equals $\mathcal{F}_{\alpha_0}(C)$ in the initial state. Its crossing support is therefore $N_{G_D}[C]$; moreover it has at most $7k + 6$ vertices.

Proof. Run the backwards selection procedure on a successful chronological trace, physically considering the successive pre-move states. In addition to its active dart set A , retain the union C of the cores already selected, beginning with the one- or two-crossing terminal core. Maintain the following invariant: in the current reverse-scan state, $A = \mathcal{F}_\alpha(C)$ and $G[C]$ is connected. It holds at the terminal state because a monogon core has one crossing and a bigon core has two adjacent crossings.

Consider a move with core Q and its unchanged footprint $F = \mathcal{F}_\alpha(Q)$ in the state immediately after it. If F is disjoint from A , the backwards algorithm omits the move. Reversing it changes no entry in A , and changes no edge incident to a dart of $D(C) \subseteq A$. Thus both parts of the invariant persist with the same A, C .

If F meets A , retain the move. Lemma 6.5 says that Q intersects or is adjacent to C . The core Q is a triangle also in the post-move state, so $G[C \cup Q]$ is connected there. Updating the active set gives $A \cup F = \mathcal{F}_\alpha(C \cup Q)$. Now reverse the move. Because $C \cup Q$ contains the whole move core, star invariance and component invariance from Lemma 6.5 give the desired invariant in the pre-move state. Induction reaches the initial state. At most three crossings are added per retained move and at most two are present in the terminal core, proving (6). The asserted identity of the full dart union follows from the same invariant.

For the last estimate, each retained move has a crossing footprint of size at most nine. At a retained step, two nonempty stars intersect. If their cores are disjoint, a joining edge places its two distinct endpoint crossings in the intersection of the crossing supports. If the cores share a crossing v , their star intersection contains the whole star of v . In a connected projection graph with more than one vertex, v has a neighbor distinct from itself, again giving at least two shared crossings. A nonempty RIII trace necessarily has at least three vertices. Thus a retained move adds at most

seven new crossing vertices to the active support. Starting with at most six in the terminal support gives $7k + 6$. Zero-length witnesses do not require this argument. $\square$

This proof uses the special exterior-port behavior of RIII, not just constant footprint size. Arbitrary rewrites inside a closed footprint could pair two exterior endpoints to one another and disconnect their old core. Such operations would invalidate the contraction argument; the theorem must not be transferred to them without checking it.

The proof also explains why unrelated births can be discarded here, but not for a completely arbitrary global target. A global predicate can require changes in many disjoint regions without having a constant-size terminal footprint. Theorem 4.3 still applies to that setting; Theorem 6.3 need not.

7 An unconditional single-exponential unlocking algorithm

The simple projection graph obtained by ignoring loops and merging parallel edges has maximum degree at most four. For nonempty one-component diagrams it is connected. We need connected *vertex sets*, not induced trees.

Lemma 7.1 (Enumerating connected regions). *In a maximum-degree-four graph with N vertices, the number of nonempty connected vertex sets of size at most s is $O(N16^s)$. They can be enumerated without repetition in polynomial workspace and with polynomial overhead per emitted set.*

Proof. For a connected set of size r , choose its least vertex as root and a canonical rooted spanning tree. A depth-first traversal of the tree is a walk of length $2(r - 1)$ in the original graph whose visited vertex set is exactly the chosen set. A fixed walk identifies its visited set, so mapping each set to its canonical traversal is injective. There are at most $N4^{2(r-1)} = N16^{r-1}$ such walks. Summing over $r \leq s$ proves the bound.

For unique enumeration, root a region at its least vertex. Maintain a selected connected set, an adjacent frontier, and a permanently excluded set. Choose the least frontier vertex, branch on inclusion or exclusion, and on inclusion add its eligible neighbors to the frontier. Vertices below the root are never eligible. Emit the singleton root once and every new selected set on an inclusion branch. Every connected set has a unique include/exclude history: its membership dictates each branch while its connectivity guarantees an available frontier path to each member. The same selected set is not re-emitted along exclusion-only branches.

A frontier or excluded vertex is adjacent to a selected vertex, so along a branch there are at most $4s$ such vertices. Every branching node emits an inclusion child; the full binary search tree has only linear overhead relative to its inclusion events and leaves. Selected sets, iterator state, and the explicit depth-first stack therefore use space polynomial in N, s , with polynomial work per output. Stop extending once the selected size reaches s . $\square$

Theorem 7.2 (Complete bounded RIII unlocking). *There is a deterministic algorithm for $\text{Unlock}_{\text{RIII}}(D, k)$ on classical knot diagrams with running time*

$$\text{poly}(N, k)2^{O(k)} \tag{7}$$

and polynomial workspace. For $k \geq 1$, one conservative explicit upper bound is

$$\text{poly}(N, k)2^{23k+11}. \tag{8}$$

It outputs a replayable witness on a positive instance and a complete negative answer only for this bounded local subproblem.

Proof. First test every RI/RII face. A positive test gives a zero-move witness. If there is none and $k = 0$, return the bounded negative answer. Otherwise put $t = 3k + 2$. Enumerate all nonempty connected vertex sets C of size at most t in the initial projection graph. For each, run the layer search through k actions, without a birth bound, permitting precisely those moves whose three-crossing core is contained in C . Keep the full ambient state and continue to use the full dart footprint for legality, commutation, predecessor incidence, and transitions. Test for a terminal reduction after each yielded layered prefix.

Every returned path consists of legal RIII moves followed by a legal test, so the algorithm is sound. For completeness, choose the connected core C given by Theorem 6.6. All moves of its retained witness are eligible. Disjoint swaps preserve their cores and full footprints, so their layer normal form is eligible as well. Theorem 3.1 guarantees that the search visits a successful endpoint. If $N \leq t$, it suffices to run once on the whole vertex set, since that set contains every possible witness core.

For complexity, there are $O(N16^t)$ regions by Lemma 7.1. A root action eligible for C is a triangular face with three distinct darts at crossings of C . There are at most $4|C|/3 \leq 4t/3$ such faces. Summing all root subsets costs at most $2^{4t/3}$. Theorem 4.3 with $d = 36$ bounds the search for each region by $\text{poly}(N, k)2^{4t/3}(36e)^k$. Consequently

$$T(N, k) \leq \text{poly}(N, k) 16^{3k+2} 2^{(4/3)(3k+2)} (36e)^k. \quad (9)$$

Since $36e < 2^7$, the exponential is at most $2^{12k+8+4k+8/3+7k} \leq 2^{23k+11}$. Counts of roots, lengths, and the factor N have been absorbed into the polynomial. The displayed base is a complexity bound, not a prediction of operational cost.

Process regions one at a time. Their enumeration has polynomial workspace by Lemma 7.1; the layer search has polynomial workspace by Theorem 4.3. This proves the space claim. $\square$

Remark 7.3 (Two safe localization modes). The supplied default region mode uses the connected core bound $3k + 2$. A second, slower theoretical variant enumerates connected full-support regions of size at most $7k + 6$ and restricts whole footprints to those regions. Both keep the ambient diagram. Using only core crossings as *read/write footprints* would be unsafe; using them as an additional eligibility restriction while still checking the full footprint is safe.

7.1 Bit complexity and implementation costs

A dart index uses $O(\log(N + 1))$ bits. The state has $4N$ entries, so its pairing array occupies $O(N \log(N + 1))$ bits. Every move changes a constant number of entries and can be specified by three dart indices. A complete revalidation of an explicit state, or even a polynomially inefficient candidate scan, contributes only to the polynomial prefactor in (7). The bound is consequently not dependent on a unit-cost assumption for arbitrarily large coordinates or coefficients.

The supplied reference code deliberately does not optimize this prefactor: it stores immutable tuples, computes faces explicitly, and uses Python sets and sorted keys. In particular, candidate generation currently may construct a temporary state to reject a degenerate reconnection. This is still polynomial work per candidate; removing that copy is a possible engineering change, not a condition needed for the theorem.

The parameter is the numerical value of k . Encoding k in binary does not turn $2^{O(k)}$ into a polynomial bound in its bit length. The claim is fixed-parameter tractability with a single-exponential parameter function, not strong polynomiality in an unbounded numerical budget.

7.2 Algorithmic skeleton

```

Unlock(D, k):
  if a legal RI/II face exists in D: return zero-length witness
  if k == 0: return NO_BOUNDED_UNLOCK
  if there is no legal first RIII: return NO_BOUNDED_UNLOCK
  t = 3*k + 2
  regions = connected initial vertex sets of size at most t
  if N <= t: regions = [all vertices]
  for C in regions:
    search legal commuting layers of total size at most k
    keep full ambient diagram; allow a iff its core is in C
    use full F(a) for independence and previous-layer contact
    use every root subset; charge all individual moves
    if a completed prefix exposes RI/II: return its certificate
  return NO_BOUNDED_UNLOCK
# A trial/region/deadline cap returns UNKNOWN, never the last line.

```

An empty initial RIII catalogue also permits immediate complete bounded failure, since no nonempty trace can begin. This shortcut is already present in the inspected production causal-search discussion [1]; it is not a new knot-type criterion.

The phrase “kernel” here denotes a bounded-support search component. We do *not* claim a many-one polynomial kernelization reducing every N -crossing instance to one equivalent $O(k)$ -crossing instance. There can be $N2^{O(k)}$ candidate regions, and external pairing data remain present.

8 Consequences for the complete recognition pipeline

The local result becomes relevant to the target complexity only when combined with an explicit policy and a controlled residual computation. Existence of a favorable but unselected sequence is not a bound on the actual implementation.

Definition 8.1 (Budgeted deterministic reduction policy). Fix deterministic choices for RI/RII exhaustion, connected-region order, layer order, and witness commitment. Start from D . Exhaust crossing- decreasing RI/RII moves. At each stalled nonempty diagram, call the exact uncapped bounded unlocking algorithm with budget g . On success, apply the returned RIII path and at least one exposed reduction, then repeat RI/RII exhaustion. On bounded failure, stop preprocessing and send the residual diagram to an exact fallback. Let r be that residual diagram’s crossing count; put $r = 0$ when the empty diagram is reached.

Lemma 8.2 (A suitable exponential fallback exists). *A classical knot diagram with r crossings can be recognized exactly in $2^{O(r)}$ bit time by explicit reduced Khovanov computation over $\mathbb{Q}$.*

Proof. There are 2^r resolutions. Each has at most $r + 1$ circles, so the number of basis generators in the unreduced complex is at most 2^{2r+1} ; reducing does not increase it. The usual merge and split maps have entries $0, 1, -1$. Explicit construction is bounded by a polynomial in r and this exponential dimension. Fraction-free elimination computes the ranks of all differential matrices.

Hadamard's determinant bound shows that their minors have bit length polynomial in the dimension and in r , hence $2^{O(r)}$. Polynomially many arithmetic operations in that dimension with these bit lengths still cost $2^{O(r)}$. The total rational homology rank is then exact. Reduced rational Khovanov homology has rank one exactly for the unknot by the unknot-detection theorem [9]. $\square$

This lemma supplies a complexity-compatible complete fallback; it does not say that a new homology implementation is included in this package. The repository already has complete exact backends. The new package contains only the local search, its verification machinery, and a proposed bridge to its mutable dart representation.

Theorem 8.3 (Run-dependent gap/core bound). *For the deterministic reduction policy above, using an exact fallback with cost $2^{O(r)}$, recognition has cost*

$$\text{poly}(N, g)2^{O(g)} + 2^{O(r)}. \quad (10)$$

In particular, a family on which the actual run has $g, r = O(\log^2 N)$ admits recognition in $N^{O(\log N)}$ time. If the policy reaches the empty diagram, it produces an unknot witness of $O(N(g + 1))$ Reidemeister moves.

Proof. Each successful episode includes a crossing-decreasing move. There are at most N such episodes, and the number of crossings during every RIII search is at most N . RI/RII exhaustion is polynomial in the explicit input size; even rebuilding after each reduction suffices for that claim. Theorem 7.2 gives at most $N \text{poly}(N, g)2^{O(g)}$, with the factor N absorbed into the polynomial. There is at most one unsuccessful final bounded call, and its cost has the same bound. Add the residual fallback cost.

If $g, r \leq C \log^2 N$, the exponential factors are $2^{O(\log^2 N)} = N^{O(\log N)}$; the polynomial prefactor does not change this form. There are at most N successful episodes, each with at most g RIII moves and a total of at most N crossing-decreasing moves over the entire run. This proves the certificate-length bound. $\square$

With fixed original dart labels retained across deletions, the positive certificate uses $O(N(g + 1) \log(N + 1))$ bits. A verifier replays the moves without searching. Full revalidation after every move is polynomial in N and certificate length; incremental verification can be faster but is not needed for this assertion.

8.1 Where the universal theorem is still missing

Nothing here proves that every unknot admits a successful RIII-only unlocking episode with $g = O(\log^2 N)$ at each stall of this policy. Nothing proves that the residual r is polylogarithmic when the bounded search fails. The method never authorizes a knottedness verdict merely from local failure, even after complete bounded exhaustion.

These are substantive geometric and policy questions, not a missing constant in the combinatorial analysis. Allowing crossing-increasing moves would require new support bounds, dynamic creation and deletion of ports, and corresponding completeness arguments. A universal hierarchy method would require its own representations and costs; it cannot be inferred from the local causal cone.

The improvement nevertheless changes a useful scale. The bounded local query is polynomial for $k = O(\log N)$ and fits the specified $N^{O(\log N)}$ budget for $k = O(\log^2 N)$, without any independent-birth promise. The old k^k factor at the latter scale contributes an additional $\log \log N$ in the exponent. Root removal additionally avoids a large N^b term when many births are possible.

9 Executable design, certificates, and integration safety

9.1 Package components

Module	Responsibility
<code>layered_search.py</code>	Generic pure-state layer enumeration, bounded roots, normal-form and backwards-cone utilities, explicit resource exceptions.
<code>dart_kernel.py</code>	Classical diagram validation, local RIII pairing patches, RI/RII tests, and reference reduction operations.
<code>connected_kernel.py</code>	Unique connected-region enumeration and the complete bounded unlocking algorithm.
<code>verify_certificate.py</code>	Search-independent certificate replay and validation of layers and terminal faces.
<code>production_adapter.py</code>	Snapshot/commit bridge retaining original production dart identities and rolling back on failure.
<code>unlock.py</code>	Command-line bounded query with explicit FOUND_UNLOCK, NO_BOUNDED_UNLOCK, and UNKNOWN statuses.
<code>audit.py</code> , <code>benchmark.py</code>	Deterministic mathematical audits and paired kernel timings, with raw JSON output.

All Python code uses the standard library. No Lean formalization or other proof-assistant artifact is claimed. The certificate verifier is independent of the *search control flow*, but shares the dart reconnection kernel. Thus it is not an independent second implementation of knot topology. The comparisons against chronological enumeration independently exercise the search logic, not the provenance of the common Reidemeister formulas.

9.2 One global copy for a disjoint layer

For a pure-state implementation, copying the $4N$ -entry pairing array after each move costs $O(N\ell)$ for a layer of width ℓ . Since footprints are disjoint, compute each constant-size patch from the common pre-layer state, allocate one copy, and apply the patches to it. The cost becomes $O(N + \ell)$ pairing-entry operations, after candidate validation. The implemented `apply_layer` uses precisely this method. It does not claim a corresponding speedup over the production mutable journal, which already avoids global copies in a different way.

If patch creation or validation fails, the original immutable state is untouched. An exception while constructing a tentative result cannot leak a partially modified input. This is useful for resource-safe research experiments even when the copying overhead is not competitive.

9.3 Certificate contract

A certificate records the input PD code, a maximum move count, a sequence of nonempty layers of three-dart face keys, and a specific terminal RI/RII face. Its format identifier is `proveit-r3-layer-certificate-v1`. The verifier checks classical one-component input validity, literal integer types, legal face identity, canonical key ordering, pairwise footprint disjointness in a layer, intersection with the preceding layer, legal transitions, move count, and the selected terminal reduction. It revalidates the diagram after every move.

The face key matters. A set of three crossing indices can identify two different triangular faces, as the repository’s explicit audit shows [1]. The new format identifies darts in the current fixed-label episode, not just crossings. The bridge maps those compact episode labels back to the original live production dart labels after previous crossing deletions.

Certificates are positive witnesses only. A completed search can provide a trustworthy local negative *computation*, but we do not include a succinct independently checkable certificate of bounded nonreachability. A timeout or exhausted subset/region allowance cannot be serialized as a negative certificate.

9.4 Safe opt-in adapter

The adapter snapshots the production object’s live crossings and pairing entries, validates the resulting compact diagram, and records the mapping to original labels. Search occurs on the isolated snapshot. Before any commit, the candidate certificate is replayed, its final pairing is checked, and the live production object is compared with the original snapshot to reject stale data.

The bridge reconstructs actual cyclic faces from verified keys; it does not trust an action’s optional payload. It journals the original live pairings and the original path length before writing. Any exception, including a cooperative cancellation during mutation, restores the pairing and truncates the path to that original length. It does not perform the exposed crossing deletion, change production trial counters, change defaults, or invent a verdict.

This is an integration proposal with unit-tested invariants, not an already integrated production patch. The full repository regression suite was not run with this bridge. Compatibility with evolving internal APIs, budget accounting, restart handling, and the existing independent trace replayer must be checked in the real checkout before enabling it.

10 Validation and measured performance

All results in this section are completed runs of the delivered code. The environment was CPython 3.13.5 on Linux x86-64, with platform string recorded in the raw benchmark JSON. No performance result here is a measurement of the complete **fastunknot** recognition pipeline.

10.1 Tests and exhaustive checks

The final local unit suite has 27 passing tests. It includes invalid parameters, caps, cancellation, the inverse-padding counterexample, canonical fork/join layers, connected-set enumeration, actual RIII face ambiguity, disjoint commutation, batch-versus-sequential transitions, certificate tampering, deleted-label mapping, stale snapshots, rollback, and rejection of a forged final state. The adapter also has a regression test ensuring that an untrusted action payload cannot be committed as an unchecked move identity.

The independent Boolean audit uses all 27 guarded one-bit toggles on three sites: choose the toggled bit and independently choose whether each other bit is untested, required zero, or required one. Each rule is an involution because its guard does not read its flipped bit. All rule sets of sizes one through three give $\binom{27}{1} + \binom{27}{2} + \binom{27}{3} = 3303$ systems. For all eight initial states and birth bounds one through three, compare chronological enumeration through depth four with the layered endpoint set. That gives 79,272 comparisons, covering 1,313,568 reference trace visits, with zero

discrepancies. Equality of endpoint sets is tested, not merely equality of a yes/no outcome on a few selected goals.

The forest formula is checked independently by truncated series convolution of $T = z(1 + T)^d$ rather than by reusing its binomial expression. There are 651 coefficient comparisons over the recorded ranges, with zero discrepancies. These finite comparisons support the implementation; Lemma 4.2, not the tests, proves the formula for all parameters.

10.2 Actual-diagram audit

Using seed 8102601, the audit generates 1,200 braid words on three through six strands, with lengths eight through 32. It retains the 248 words whose closures validate as knots, then performs deterministic RI/RII exhaustion. On eligible resulting states it checks individual RIII supports and moves, disjoint pairs, and bounded endpoint sets. It also runs random legal RIII traces of length at most eight, extracts the backwards cone of available terminal reductions, replays the retained subsequence and its layer form, and checks both support and core connectivity in the initial projection graph. The same 1,000 cases check the exact identity of the cone’s full dart union with the initial dart star of its retained core union, and the corresponding closed-neighborhood identity on crossings.

Audit item	Completed count
Validated knot closures	248
Bounded actual-diagram endpoint comparisons	180
RIII action/support/inverse checks	821
Disjoint actual-diagram commutation checks	543
Terminal-cone extraction and initial-connectivity checks	1,000
Core connectivity and initial-star identities (same cones)	1,000
Nonempty cones among those checks	1,000
Retained positive certificate files	10
Discrepancies in these checks	0

The largest observed footprint has 18 darts and nine crossing vertices, attaining both proved local upper bounds. The largest observed site incidence is five and the largest observed next-parent candidate count is nine; these smaller observations do not replace the uniform proved bounds eight and 36. The ten certificates in `artifacts/` are nonempty positive witnesses. They demonstrate replay, not difficult unknot recognition or a lower bound on minimal unlocking depth.

10.3 Paired kernel timings, including negative results

We compare a pure-state implementation of the old birth-front normal form with the new commuting-layer enumerator. Both use the same transition kernel; the baseline omits the production code’s optional inverse pruning. Every trace prefix through the specified depth is exhausted. Goal tests do not terminate these timing runs early. This measures interleaving work, not the time to find a successful reduction, and not full recognition. An identical baseline control is timed in each round. There is one untimed warmup of the distinct algorithms, followed by five paired rounds with randomized arm order, seed 8102602.

The knot cases are the closures of $(\sigma_1\sigma_2)^q$ for $q \in \{4, 7, 10\}$. Their purpose is to provide genuine diagram operations with controlled independent regions, not to represent a hard-knot corpus. The six-bit cases are explicitly abstract systems. The paired ratio in Table 1 is the median of baseline time divided by layer time *within each round*; it is not in general the quotient of the two reported marginal medians.

Case	k	Old traces	Layers	Old ms	New ms	Paired ratio
Six bits	1	6	6	0.010	0.018	0.549
Six bits	6	1,653	923	4.526	4.614	0.952
Six bits	8	15,327	3,002	35.911	15.747	2.261
$(\sigma_1\sigma_2)^4$	1	8	8	0.102	0.109	0.925
$(\sigma_1\sigma_2)^4$	4	664	664	8.918	9.963	0.896
$(\sigma_1\sigma_2)^7$	4	6,069	4,928	85.497	86.583	0.980
$(\sigma_1\sigma_2)^{10}$	4	18,095	14,305	291.492	307.004	0.952

Table 1: Exhaustive kernel timings. “Layers” counts yielded layered trace prefixes, not individual layer objects. Ratios above one favor the new code. These are not end-to-end recognition speedups.

For the 14- and 20-crossing diagram cases, the new normal form visits about 18.8% and 20.9% fewer trace prefixes, respectively. However, layer construction and subset testing erase that advantage in the measured wall-clock times. The actual-diagram paired ratios range from 0.896 to 0.980: there is no demonstrated knot-kernel speedup in these runs. The abstract six-bit depth-eight case does improve, but promoting its 2.261 ratio to a recognition claim would be misleading.

The identical-control paired ratios range from approximately 0.963 to 1.024 across these cases. This is a small observational noise check, not a confidence interval or a statistical performance guarantee. Very short measurements, especially the depth-one Boolean row, are intrinsically sensitive to scheduling and timer variation. All per-round times and arm orders are retained in `data/benchmark.json`; counts are deterministic.

The study therefore supports a theoretical improvement and a reduction of some duplicate work, *not* a default implementation switch. The connected-region algorithm itself has not been subjected to a large end-to-end performance comparison. Its conservative constant and region multiplicity may dominate at practical parameter values.

11 Limits, proof obligations, and failure modes

A local terminal is essential. The backwards-cone proof starts with one fixed-size terminal footprint. It cannot be used without modification for a global invariant comparison, full homology calculation, or a target depending on many separated regions. In those cases a birth or terminal-support parameter may remain.

The full footprint is essential. The three flipped crossings alone are not a safe read/write support. Partner darts can be read or changed. The core-region algorithm uses core inclusion only to restrict its action catalogue; every independence, legality, and update check still uses $D \cup \alpha(D)$, including all remote endpoint darts. Treating the core as the footprint would destroy the proof.

Graph evolution must be controlled. Being connected in the graph immediately before each move does not, by itself, imply that a union is connected initially. The confinement lemma also

requires that every changed edge have both endpoints inside the move’s connected support. It is proved for the retained trace, after irrelevant actions are removed; ignoring that order of reasoning leaves a gap.

A length bound is not a layer-count bound. Charging every nonempty parallel layer one unit could hide arbitrarily many actions. All statements and code charge the sum of layer sizes. The certificate verifier checks that sum independently.

Finite caps change the specification. The complete theorems assume the search exhausts its domain when no witness exists. In an operational recognizer, resource caps should be expected. They produce UNKNOWN and preserve the existing complete fallback. The package’s `SearchExhausted` exception is not a negative recognition result.

Local choices need not compose optimally. Selecting one available reduction can change later unlocking depths. The gap/core theorem explicitly concerns the actual deterministic policy. It cannot substitute the existence of a different globally favorable reduction sequence. The distinction is also consistent with the subtleties of greedy moves in global untangling problems [7].

Testing is not proof or complete integration. The tests check concrete implementations and many instances. They neither prove all geometrical claims nor demonstrate compatibility with every current production pathway. The mathematical claims are justified by the preceding proofs. The package does not report a rerun of the repository’s full test suite and does not report a new complete recognizer.

Priority and external review. The layer machinery builds on classical partial commutation. The connected initial-support theorem and its combination with the dart bounds are presented as this contribution’s main advances relative to the inspected repository. The literature review is not a proof that no equivalent parameterized local-rewrite theorem has appeared elsewhere. The arguments should receive independent mathematical review before being described as an established breakthrough.

12 Further research questions and concrete work packages

12.1 Reduce the single-exponential base without weakening completeness

Question 1: Can connected heaps be enumerated directly? The factor 16^{3k+2} comes from choosing a connected core region before searching its layers. Different regions can contain the same causal trace. A direct exploration of connected terminal-rooted dependency structures may avoid this multiplicity. The difficulty is that the terminal face is not necessarily a face in the initial diagram. A useful result would be an injective code for terminal-rooted dynamic heaps with NC^k codes and a practically smaller C , including an explicit decoder.

Question 2: Is the next-parent bound 36 sharp? The estimate counts four corners at each of nine support crossings and ignores triangle compatibility, over/under parity, and the fact that six of the vertices are exterior neighbors. Determine the maximum number of legal footprints meeting one post-flip footprint in a realizable one-component spherical map. An exhaustive local

rotation-system catalogue could conjecture a better value, but a uniform proof must handle all external identifications rather than only generic six-ended tangles.

Question 3: Can region enumeration exploit triangle coverage? The core union has at most $3k + 2$ crossings, but it is not an arbitrary connected set: it is covered by the cores of retained triangles and one terminal core. Encode that additional structure directly, while allowing the triangles to become faces only after earlier operations. Determine whether this yields fewer candidates than the generic 16^{3k+2} region bound. Bounds based only on a static catalogue of initial triangular faces would miss dynamically created faces.

12.2 Turn reduced interleavings into measured speed

Question 4: Which subset generator is best for the current dart engine? The reference generator enumerates combinations and then checks compatibility. A prefix-pruned independent-set generator in the conflict graph could reject overlaps earlier. A production benchmark must compare against the existing journal-based search, preserve inverse semantics, and include all candidate-generation overhead. Node counts alone are not a sufficient acceptance criterion.

Question 5: Can certified state sharing reuse work between regions? Two regions often induce identical eligible actions along a prefix. Cache keys might include the live pairing on an explicitly closed support, the previous layer, remaining depth, and admissibility information. Prove a sufficient key before sharing results. Physical endpoint equality by itself is not enough for every restricted search. A cache should also have a polynomial-space mode or a separately stated space bound.

Question 6: Can layer batching be implemented with the mutable production journal? The current reference code makes one global copy per layer. The existing production engine already journals local writes. A joint layer transaction could register all disjoint patches once, perform a localized terminal scan, and roll back atomically. Required tests include injected exceptions at every mutation point, deleted-dart identities, and replay using the repository’s existing verifier.

12.3 Bridge the local theorem to stronger recognition classes

Question 7: What families have polylogarithmic policy gap and residual core? Measure the actual quantities entering Theorem 8.3, not only successful traces found by a different policy. Natural first studies include diagrams with controlled braid run structure, small connected-sum cores, and the project’s stored hard unknot survivors. A theorem should specify the chosen normalizations and reduction order, and should bound the total preprocessing that obtains its promised representation.

Question 8: Can limited crossing increases be incorporated? Introduce an additional parameter for created crossings or for a bounded height above the initial crossing count. Track provenance of new ports and local terminal ancestors. Does an initial-support argument survive when graph vertices are created, perhaps by attaching them to the old ports that created them? A successful result must bound both the number of created sites and the encoding of their attachment histories.

Question 9: Can a policy-independent residual theorem be proved? Instead of requiring every selected episode to be favorable, ask whether all maximal sequences of reductions with a specified local completeness budget leave a controlled core. Alternatively, identify a efficiently verifiable potential that makes one policy provably safe. Greedy crossing count alone does not

provide such a theorem.

Question 10: Is there a useful terminal-support hierarchy? The cone lemma permits a terminal support larger than a single RI/RII move if its size is explicitly parameterized. One could target a certified braid-kernel shortening, a small disk simplification, or a structured multi-move certificate. The intended target must be recognized locally with controlled support; importing a global invariant oracle without its cost defeats the bound.

12.4 Relate local search to hierarchy and complexity theory

Question 11: Can dependency layers organize hierarchy operations? Lackenby’s hierarchy approach is not a sequence of the fixed-site RIII moves used here. Nevertheless, one may ask whether an encoded hierarchy admits certified independent operations with bounded interaction neighborhoods. Before transferring the forest count, prove an appropriate representation-size bound, conservative read/write supports, dynamic commutation, and the analogue of d . Merely drawing an analogy with parallelism would not supply an algorithm.

Question 12: What parameterized lower bounds match this local problem? Study the precise bounded unlocking language, with no crossing increases and only one local terminal reduction. Determine whether $2^{o(k)} \text{poly}(N)$ is plausible under standard hypotheses. Reductions must preserve classical one-component planarity and the exact allowed moves. Existing hardness for whole-diagram defect does not automatically supply this lower bound.

Question 13: Can the proofs be formalized modularly? A useful formalization order is: finite-site exchange contracts; occurrence heights and linear extensions; bounded-slot forest injection; finite graph component confinement; dart footprint lemmas; then the full algorithmic count. This separates generic combinatorics from Reidemeister topology. The certificate format and finite audit catalogue provide concrete test objects, not replacements for these formal proofs.

12.5 Immediate integration decision

Retain the current production default. Add the new search as an explicitly experimental bounded-unlocking mode after replay and budget integration are verified in a complete checkout. Measure paired end-to-end recognition on the project’s actual hard corpus, with unchanged complete fallbacks, identical global budgets, randomized arm order, and negative results retained. Promote a default only on that evidence; preserve the proved complete mode separately from heuristic early dispatch.

13 Conclusion

Two independently useful effects emerge. Commuting layers remove the k^k interleaving overhead from the birth-bounded local search. A terminal ancestor cone, together with initial-component and core-contraction invariants, then removes the birth-choice factor for the specific RI/RII unlocking problem. The result is a complete deterministic single-exponential-parameter search, with explicit local constants and polynomial workspace.

The new bottleneck is therefore not enumeration of all chronological interleavings of a short unlocking episode. It is the geometric size of an episode needed by the selected simplification policy, the size of the residual core when that policy stalls, and practical constant factors. The gap/core theorem states exactly what would suffice to reach $N^{O(\log N)}$ for the complete pipeline. The included

code and audits establish a reproducible starting point for that next step without misreporting a general recognition theorem or an unmeasured speedup.

A Reproduction and integration instructions

From the package root, a Unix-like shell can run:

```
export PYTHONPATH=src
python -m unittest discover -s tests -v
python src/audit.py
python src/benchmark.py --rounds 5
python src/verify_certificate.py artifacts/witness_00.json
python src/unlock.py artifacts/witness_00.json --depth 4 \
    --max-trials 100000 --certificate /tmp/replayed-unlock.json
sh build.sh
```

The input accepted by `unlock.py` is either a PD object or an Artin braid object. A certificate file includes a PD field and is therefore also a valid query input. The example above searches again; it does not assume the certificate’s listed path is minimal. Omit caps only when complete bounded exhaustion is intended. A zero cap means zero allowance, not an unlimited search. Running audits or benchmarks regenerates their JSON outputs, including local timings; the shipped outputs record the run reported in this article.

`README.md` provides a package map and status summary. `INTEGRATION.md` gives the production adapter contract and the recommended opt-in integration sequence. `SOURCES.json` records source paths, revisions, blob hashes, and adaptation scope. The archive contains no third-party preprint PDF or font file, and no complete production checkout. `MANIFEST.sha256` covers the delivered files. The article is self-contained LaTeX with an inline bibliography, so no external bibliography download is needed to rebuild it.

B Proof-dependency and evidence ledger

Statement	Mathematical dependency	Computational evidence
Dynamic normal form	Finite occurrence poset and stable disjoint exchange contracts	Boolean and diagram endpoint comparisons
Forest budget	Deterministic parent/slot assignment; formal coefficient extraction	651 convolution comparisons
18-dart, nine-crossing bound	Six internal triangular-side darts and unchanged exterior endpoints	821 action/support checks
36 successor bound	Post-layer pairing closure and four corners per crossing	Observed maximum nine; not used as proof
Terminal cone	Backwards support test and exchanges of omitted actions	1,000 retained-trace replays
Initial connectedness	Component confinement for full supports; core contraction and star invariance for connected $3k + 2$ cores	1,000 core connectivity and initial-star equality checks

Statement	Mathematical dependency	Computational evidence
Single-exponential unlocking	Region count, local root catalogue, forest budget	Executable exact region/layer search
Gap/core recognition bound	At most N successful episodes and an exact $2^{O(r)}$ fallback	No uniform gap/core measurements or proof claimed
Production adapter	Snapshot validation, verified keys, state-state check, journaled commit	Local unit tests only; full production suite not run

References

- [1] V. Reshetnikov’s ProveIt repository, *Causal RIII search: review and unambiguous certificates*, Topology/UnknotRecognition/synthesis/causal_r3.tex. Inspected 8 October 2026; blob b5b7b3f01db47d358df2a7b9889865c635a42044. <https://github.com/VladimirReshetnikov/ProveIt>.
- [2] ProveIt contributors, fastunknot/causal_r3.py, Topology/UnknotRecognition/fast/, blob 19599a59b461f74c0151acb40c3658a6a61656b7. Inspected 8 October 2026. <https://github.com/VladimirReshetnikov/ProveIt>.
- [3] ProveIt contributors, fastunknot/simplify.py, inspected lines 1–310 at revision 34f2689e96e04a13a32d133173b5e41a34320086; blob 703975603890b0481a31d7368502f48fdda22375. <https://github.com/VladimirReshetnikov/ProveIt>.
- [4] ProveIt contributors, fastunknot/diagram.py, inspected lines 1–220 at revision 34f2689e96e04a13a32d133173b5e41a34320086; blob 8717f530cc2847492858da46ac4ac40a2288b7ee. <https://github.com/VladimirReshetnikov/ProveIt>.
- [5] G. X. Viennot, *Heaps of pieces, I: Basic definitions and combinatorial lemmas*, in *Combinatoire énumérative*, Lecture Notes in Mathematics 1234, Springer, 1986. <https://doi.org/10.1007/BFb0072524>.
- [6] C. Krattenthaler, *The theory of heaps and the Cartier–Foata monoid*, author manuscript. <https://www.mat.univie.ac.at/~kratt/artikel/heaps.pdf>.
- [7] C. Legrand-Duchesne, A. Rai, and M. Tancer, *Parameterized complexity of untangling knots*, arXiv:2111.05001v1, 2021. <https://arxiv.org/abs/2111.05001>.
- [8] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026, especially Section 9. <https://arxiv.org/abs/2607.23350>.
- [9] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publications Mathématiques de l’IHÉS 113 (2011), 97–208. <https://doi.org/10.1007/s10240-010-0030-y>.
- [10] D. Bar-Natan, *Fast Khovanov homology computations*, Journal of Knot Theory and Its Ramifications 16 (2007), 243–255; arXiv:math/0606318. <https://www.math.utoronto.ca/~drorbn/papers/FastKh/>.